KU LEUVEN

FACULTEIT INGENIEURSWETENSCHAPPEN

Dynamic parallelism and asynchronous operation
Lab 5

Geavanceerde computerarchitectuur

Wouter Fouquet
Florian Vandenbogaerde
Master elektronica ICT
Academiejaar 2024-2025

CONTENTS

Exercise **1**
0.1 Sync vs async execution comparison 1
0.2 The exercise in figure form 1
0.3 Pointers 2

1 Methodology **3**
1.1 Introduction 3
1.2 Measurements 3

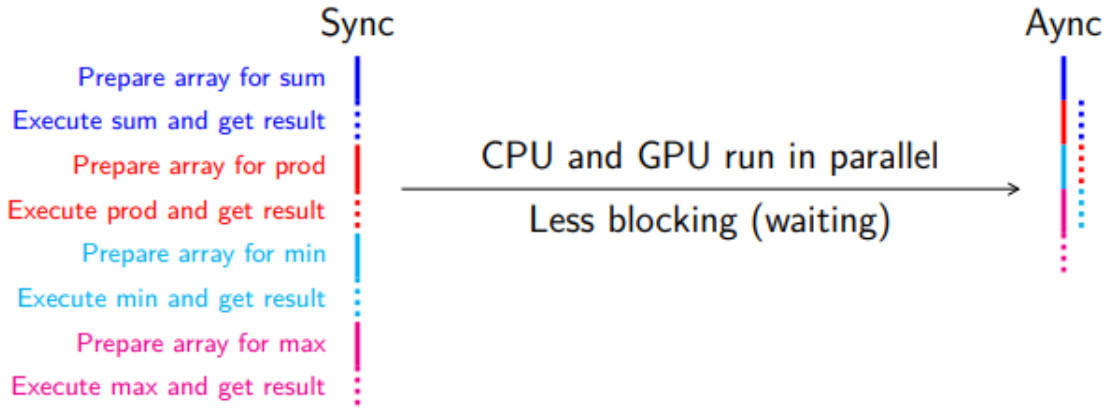
0.1 Sync vs async execution comparison

We will revisit reduction:

- Perform sum, prod, min, and max
- Process 4 independent arrays
- Usa a for loop to generate each of the arrays
- Approach 1: do all of this in sequence, synchronously
- Approach 2: do array generation while the previous data is being asynchrounously on the GPU
- Compare the total execution time (including CPU code)

0.2 The exercise in figure form

On Figure 1 the exercise in figure form can be found.



Figuur 1: Synchronous vs asynchronous execution

0.3 Pointers

- You already have the reduction kernel from exercise 2
- Implement summation, multiplication, and minimum detection
- Can use the same kernel and change only the computation part see below (Figure 2) (or point to a different __device__ function each time)

```
__global__ foo(...){
    ...
    #if KERNEL==0
    #   val = ... + ...
    #else
    #   val = ... * ...
    #endif
}
```

Figuur 2: Use same kernel and change only the computation part

- Implement a workflow where you define and populate an array on the CPU, before migrating it to the GPU and executing the kernel function.
- Do this 4 times, once for each operation: sum, prod, min, and max.
- Perhaps make 2 CPU functions - one which calls the kernel(s) synchronously, one after the other, and one that calls them while preparing the data for the next kernel.
- Call the two functions from main() and get the execution time using the chrono library (see cheatsheet).
- Make sure to synchronize before stopping the timer.

1.1 Introduction

In this report, we investigate the performance gains achieved by executing operations asynchronously, as opposed to synchronously. The primary objective is to analyze the impact of asynchronous execution on processing time, and ultimately, performance. Asynchronous operations allow multiple tasks to overlap in execution, potentially leading to significant reductions in overall execution time, especially in scenarios with parallelizable workloads.

The example scenario used in our analysis is depicted in Figure 1. This example will serve as the basis for comparing the two execution models and demonstrating the performance improvements that can be achieved by adopting an asynchronous approach.

1.2 Measurements

In the following measurements, we analyze the performance gains of asynchronous execution over synchronous execution for different array sizes. The results, shown in Table 1.1, illustrate how varying array sizes affect execution time and demonstrate the efficiency of asynchronous operations. This analysis is carried out by performing four different operations—sum, product, minimum, and maximum—in a single execution cycle.

For each test, the timer is started, and all four operations are performed consecutively, either synchronously or asynchronously. In the synchronous approach, each kernel executes fully before the next one begins, and the CPU is blocked during each kernel execution. In contrast, the asynchronous execution allows the kernels to be launched in parallel, enabling overlapping computation and data transfer for subsequent operations. The timer is stopped only after all operations are completed, providing a comparison of the total execution time for both methods.

By comparing the execution times of these two approaches for each operation, we can assess the performance benefits gained by allowing asynchronous execution to overlap computations. The goal is to highlight how different array sizes and operations impact the efficiency of the asynchronous approach and whether this method provides significant time savings over traditional synchronous execution.

The code snippet can be found in attachment 1.1

Based on the results presented in Table 1.1, we can observe that asynchronous execution consistently shows improved performance compared to synchronous execution across various array sizes.

Array size	Number of Threads	Number of Blocks	Synchronous execution [ms]	Asynchronous execution [ms]
4	4	1	787,87	839,57
16	16	1	816,64	893,42
128	128	1	802,81	898,73
1024	1024	1	1227,05	1062,24
4096	1024	4	791,96	706,30
8192	1024	8	1006,06	903,65
16384	1024	16	1028,44	992,31

Tabel 1.1: Comparison of execution times between synchronous and asynchronous execution for different array sizes.

For smaller array sizes (such as 4 and 16 elements), the difference in execution times is relatively minor, with asynchronous execution being slightly slower in some cases. However, as the array size increases (e.g., for 4096, 8192, and 16384 elements), the performance gain from asynchronous execution becomes more pronounced. This is especially noticeable in cases where the number of blocks is increased, such as with the 4096-element array, where asynchronous execution reduces the execution time significantly compared to the synchronous version.

These results align with our expectations, demonstrating that as the workload increases, asynchronous execution offers better utilization of resources and allows for overlapping computation with data transfers, leading to overall reduced execution times.

CONCLUSION

In this report, we have analyzed the performance benefits of asynchronous execution over synchronous execution for parallelizable operations, using various array sizes as test cases. The results, as presented in Table 1.1, indicate that asynchronous execution offers significant performance improvements, particularly when dealing with larger array sizes.

While the difference in execution times between synchronous and asynchronous execution was minor for smaller arrays (e.g., 4 and 16 elements), the gap widened as the array size increased. Asynchronous execution demonstrated notable advantages in cases where the number of blocks was higher, such as with the 4096-element array. These results confirm that asynchronous execution allows better resource utilization by enabling parallelism and overlapping computation with data transfer, which ultimately reduces the overall execution time.

Overall, our findings align with the theoretical expectations for asynchronous execution, highlighting its potential for improving performance in scenarios where tasks can be executed concurrently. This approach becomes increasingly beneficial as the size and complexity of the workload grow, making it a powerful tool for optimizing performance in parallel computing applications.

Listing 1.1: Code snippet for synchronous vs asynchronous execution

```

#include <stdio.h>
#include <cuda.h>
#include <limits.h>  // INT_MIN & INT_MAX
#include <chrono>

using namespace std;
using namespace chrono;

#define N 4096                          //ARRAY SIZE
#define BLOCK_SIZE 1024                 //NUMBER OF THREADS
#define NUM_BLOCKS (N / BLOCK_SIZE)  //NUMBER OF BLOCKS

// Genereer array op CPU
int* generateArray() {
    int* arr = (int*)malloc(N * sizeof(int));
    for (int i = 0; i < N; i++) {
        arr[i] = i;
    }
    return arr;
}

__global__ void addValue(int* d_in, int* d_out, int value) {
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    if (index < N) {
        d_out[index] = d_in[index] + value;
    }
}

__global__ void multiplyValue(int* d_in, int* d_out, int value) {
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    if (index < N) {
        d_out[index] = d_in[index] * value;
    }
}

__global__ void maxReduceKernel(int* d_in, int* d_out) {
    __shared__ int sharedMax[BLOCK_SIZE];

    int tid = threadIdx.x;
    int globalIdx = blockIdx.x * blockDim.x + tid;

    sharedMax[tid] = (globalIdx < N) ? d_in[globalIdx] : INT_MIN;
    __syncthreads();

    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sharedMax[tid] = max(sharedMax[tid], sharedMax[tid + s]);
        }
        __syncthreads();
    }

    if (tid == 0) {
        d_out[blockIdx.x] = sharedMax[0];
    }
}

__global__ void minReduceKernel(int* d_in, int* d_out) {
    __shared__ int sharedMin[BLOCK_SIZE];

```

```
    int tid = threadIdx.x;
    int globalIdx = blockIdx.x * blockDim.x + tid;

    sharedMin[tid] = (globalIdx < N) ? d_in[globalIdx] : INT_MAX;
    __syncthreads();

    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sharedMin[tid] = min(sharedMin[tid], sharedMin[tid + s]);
        }
        __syncthreads();
    }

    if (tid == 0) {
        d_out[blockIdx.x] = sharedMin[0];
    }
}


void synchronizedExecution() {
    // --- SUM ---
    int* sum_in = generateArray();
    int* sum_out = (int*)malloc(N * sizeof(int));

    int *d_sum_in, *d_sum_out;
    cudaMalloc(&d_sum_in, N * sizeof(int));
    cudaMalloc(&d_sum_out, N * sizeof(int));

    // --- MULTIPLICATION ---
    int* mul_in = generateArray();
    int* mul_out = (int*)malloc(N * sizeof(int));

    int *d_mul_in, *d_mul_out;
    cudaMalloc(&d_mul_in, N * sizeof(int));
    cudaMalloc(&d_mul_out, N * sizeof(int));

    // --- MINIMUM ---
    int* min_in = generateArray();
    int min_out;
    int *d_min_in, *d_min_out;
    cudaMalloc(&d_min_in, N * sizeof(int));
    cudaMalloc(&d_min_out, sizeof(int));

    // --- MAXIMUM ---
    int* max_in = generateArray();
    int max_out;
    int *d_max_in, *d_max_out;
    cudaMalloc(&d_max_in, N * sizeof(int));
    cudaMalloc(&d_max_out, sizeof(int));


    // --- SUM ---
    cudaMemcpy(d_sum_in, sum_in, N * sizeof(int), cudaMemcpyHostToDevice);
    addValue<<<NUM_BLOCKS, N>>>(d_sum_in, d_sum_out, 5);
    cudaDeviceSynchronize();
    cudaMemcpy(sum_out, d_sum_out, N * sizeof(int), cudaMemcpyDeviceToHost)
        ;


    // --- MULTIPLICATION ---
    cudaMemcpy(d_mul_in, mul_in, N * sizeof(int), cudaMemcpyHostToDevice);
```

```
121     multiplyValue<<<NUM_BLOCKS, N>>>(d_mul_in, d_mul_out, 5);
122     cudaDeviceSynchronize();
123     cudaMemcpy(mul_out, d_mul_out, N * sizeof(int), cudaMemcpyDeviceToHost)
            ;
124
125
126     // --- MINIMUM ---
127     cudaMemcpy(d_min_in, min_in, N * sizeof(int), cudaMemcpyHostToDevice);
128     minReduceKernel<<<NUM_BLOCKS, N>>>(d_min_in, d_min_out);
129     cudaDeviceSynchronize();
130     cudaMemcpy(&min_out, d_min_out, sizeof(int), cudaMemcpyDeviceToHost);
131
132
133     // --- MAXIMUM ---
134     cudaMemcpy(d_max_in, max_in, N * sizeof(int), cudaMemcpyHostToDevice);
135     maxReduceKernel<<<NUM_BLOCKS, N>>>(d_max_in, d_max_out);
136     cudaDeviceSynchronize();
137     cudaMemcpy(&max_out, d_max_out, sizeof(int), cudaMemcpyDeviceToHost);
138
139
140     // --- Free memory ---
141     free(sum_in);
142     free(sum_out);
143     free(mul_in);
144     free(mul_out);
145     free(min_in);
146     free(max_in);
147     cudaFree(d_sum_in);
148     cudaFree(d_sum_out);
149     cudaFree(d_mul_in);
150     cudaFree(d_mul_out);
151     cudaFree(d_min_in);
152     cudaFree(d_min_out);
153     cudaFree(d_max_in);
154     cudaFree(d_max_out);
155 }
156
157
158
159 void asynchronizedExecution(){
160     //declare streams
161     cudaStream_t stream1, stream2;
162     cudaStreamCreate(&stream1);
163     cudaStreamCreate(&stream2);
164
165     // --- SUM ---
166     int* sum_in = generateArray();
167     int* sum_out = (int*)malloc(N * sizeof(int));
168
169     int *d_sum_in, *d_sum_out;
170     cudaMalloc(&d_sum_in, N * sizeof(int));
171     cudaMalloc(&d_sum_out, N * sizeof(int));
172
173     // --- MULTIPLICATION ---
174     int* mul_in = generateArray();
175     int* mul_out = (int*)malloc(N * sizeof(int));
176
177     int *d_mul_in, *d_mul_out;
178     cudaMalloc(&d_mul_in, N * sizeof(int));
179     cudaMalloc(&d_mul_out, N * sizeof(int));
180
```

```
    // --- MINIMUM ---
    int* min_in = generateArray();
    int min_out;
    int *d_min_in, *d_min_out;
    cudaMalloc(&d_min_in, N * sizeof(int));
    cudaMalloc(&d_min_out, sizeof(int));

    // --- MAXIMUM ---
    int* max_in = generateArray();
    int max_out;
    int *d_max_in, *d_max_out;
    cudaMalloc(&d_max_in, N * sizeof(int));
    cudaMalloc(&d_max_out, sizeof(int));



    // --- SUM ---
    cudaMemcpyAsync(d_sum_in, sum_in, N * sizeof(int),
        cudaMemcpyHostToDevice, stream1);
    addValue<<<NUM_BLOCKS, N, 0, stream2>>>(d_sum_in, d_sum_out, 5);

    cudaMemcpyAsync(sum_out, d_sum_out, N * sizeof(int),
        cudaMemcpyDeviceToHost, stream2);
    cudaStreamSynchronize(stream1);
    cudaStreamSynchronize(stream2);

    // --- MULTIPLICATION ---
    cudaMemcpyAsync(d_mul_in, mul_in, N * sizeof(int),
        cudaMemcpyHostToDevice, stream1);
    multiplyValue<<<NUM_BLOCKS, N, 0, stream2>>>(d_mul_in, d_mul_out, 5);

    cudaMemcpyAsync(mul_out, d_mul_out, N * sizeof(int),
        cudaMemcpyDeviceToHost, stream2);
    cudaStreamSynchronize(stream1);
    cudaStreamSynchronize(stream2);

    // --- MINIMUM ---
    cudaMemcpyAsync(d_min_in, min_in, N * sizeof(int),
        cudaMemcpyHostToDevice, stream1);
    minReduceKernel<<<NUM_BLOCKS, N, 0, stream2>>>(d_min_in, d_min_out);

    cudaMemcpyAsync(&min_out, d_min_out, sizeof(int),
        cudaMemcpyDeviceToHost, stream2);
    cudaStreamSynchronize(stream1);
    cudaStreamSynchronize(stream2);

    // --- MAXIMUM ---
    cudaMemcpyAsync(d_max_in, max_in, N * sizeof(int),
        cudaMemcpyHostToDevice, stream1);
    maxReduceKernel<<<NUM_BLOCKS, N, 0, stream2>>>(d_max_in, d_max_out);

    cudaMemcpyAsync(&max_out, d_max_out, sizeof(int),
        cudaMemcpyDeviceToHost, stream2);
    cudaStreamSynchronize(stream1);
    cudaStreamSynchronize(stream2);


    // --- Free memory ---
    free(sum_in);
    free(sum_out);
    free(mul_in);
```

```
    free(mul_out);
    free(min_in);
    free(max_in);
    cudaFree(d_sum_in);
    cudaFree(d_sum_out);
    cudaFree(d_mul_in);
    cudaFree(d_mul_out);
    cudaFree(d_min_in);
    cudaFree(d_min_out);
    cudaFree(d_max_in);
    cudaFree(d_max_out);

}



int main() {
    printf("Number of Threads per block: %d\n", BLOCK_SIZE);
    printf("Number of blocks: %d\n", NUM_BLOCKS);

    // Warm-up runs (against overhead and cache)
    synchronizedExecution();
    asynchronizedExecution();
    cudaDeviceSynchronize();

    // Variabelen voor tijdsmetingen
    long long totalSynchronizedTime = 0;
    long long totalAsynchronizedTime = 0;

    const int num_runs = 100;

    for (int i = 0; i < num_runs; i++) {
        auto startSynchronized = high_resolution_clock::now();
        synchronizedExecution();
        cudaDeviceSynchronize();
        auto stopSynchronized = high_resolution_clock::now();
        totalSynchronizedTime += duration_cast<microseconds>(
            stopSynchronized - startSynchronized).count();

        auto startAsynchronized = high_resolution_clock::now();
        asynchronizedExecution();
        cudaDeviceSynchronize();
        auto stopAsynchronized = high_resolution_clock::now();
        totalAsynchronizedTime += duration_cast<microseconds>(
            stopAsynchronized - startAsynchronized).count();
    }

    double avgSynchronized = totalSynchronizedTime / static_cast<double>(
        num_runs);
    double avgAsynchronized = totalAsynchronizedTime / static_cast<double>(
        num_runs);

    printf("Average Synchronized Execution Time: %.2f microseconds\n",
        avgSynchronized);
    printf("Average Asynchronized Execution Time: %.2f microseconds\n",
        avgAsynchronized);

    return 0;
}
```